

T-HEAD

Maintain CPU/IO TLBs with sfence.vma for RSIC-V

Guo Ren <guoren@linux.alibaba.com>

PPT link (anyone could comment):
https://docs.google.com/presentation/d/1sc295JznVAfDlPieAqzjcyUkcHnNFQsK8FFqdoCY854/edit?usp=sharing

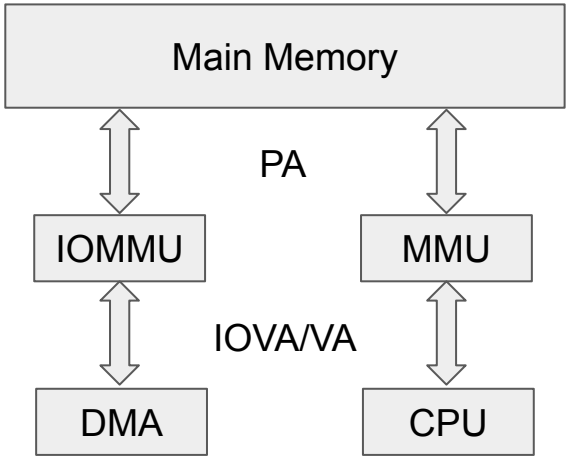
Agenda

- Quick review concepts of linux iommu
- Introduce a broadcast TLB invalidate instruction for RISC-V (sfence.vma broadcast mode)
- Introduce a light implementation of IOMMU with broadcast sfence.vma
- Is it suitable for millions of devices (PCI ATS & virtualization) ?

Concepts Review

Basic Functions:

- DMA device can directly use virtual address without the need to be contiguous in physical memory
- Solving the addressing problems of devices with limited address spaces
- Devices are isolated in different address space and protected from malicious devices or firmwares
- **Support all kinds of virtual memory method in linux ?**
  (file/anonymous mapping, swap, migration, transparent huge page ...)



Address & Address-Space Concept

- VA - Virtual address (CPU)
- PA - Physical address
- IOVA - Virtual address used by DMA devices (IOMMU translate IOVA to PA)

- ASID - Used by CPU to identify different VAs' address space in TLB
- IOASID - Used by IOMMU to identify different IOVAs' address space in IOTLB

VA + ASID => PA <= IOVA + IOASID

Address & Address-Space Concept

- VA - Virtual address (CPU)
- PA - Physical address
-

- ASID - Used by CPU to identify different VAs' address space in TLB
- IOASID - Used by IOMMU to identify different IOVAs' address space in IOTLB

VA + ASID => PA <= VA + IOASID

Linux IOMMU Concepts

- Device (single device granularity isn't possible for IOMMU)
- Group (devices are grouped for isolation in hardware topology)
- Domain (Isolation of address space)
- Compatible with DMA APIs:

```
static const struct dma_map_ops iommu_dma_ops = {
        .alloc          = iommu_dma_alloc,
        .free           = iommu_dma_free,
        .mmap                   = iommu_dma_mmap,
        .get_sgtable    = iommu_dma_get_sgtable,
        .map_page       = iommu_dma_map_page,
        .unmap_page             = iommu_dma_unmap_page,
        .map_sg                 = iommu_dma_map_sg,
        .unmap_sg       = iommu_dma_unmap_sg,
...
```

Linux VFIO Memory Map Concept

- System Administrator
  - Unbind original driver of the devices
  - Bind vfio-platform/pci drivers to the devices
  - Determine iommu group number of the devices

- User VFIO driver
  - Create container
  - Set groups to the container
  - In that container, use ioctl:VFIO_IOMMU_MAP_DMA map IOVA
  - ***Ideally device can directly use normal memory (anonymous_mapping, file_mapping, COW, swap to disk/zram, migration, transparent huge page …)***

Terminology Puzzle

At last year KVM forum, Jacob mentioned the Terminology puzzle between Intel Vt-d and Arm SMMU

Intel Vt-d	Arm SMMU
PASID	SubstreamID
PASID table	Context descriptor table
PCI requestID	StreamID
Device context table	Stream table (entry)
1th & 2th level	Stage 1 & 2 translation

Maybe, we could add 3th column next year :)

sfence.vma broadcast mode

Hardware broadcast TLB invalidate

Agenda

- Introduce S1/S2.PGD.PPN as unique identifiers to connect different address translation systems for SVM
- Introduce a async TLB invalidate mechanism for PCI ATS usage scenarios

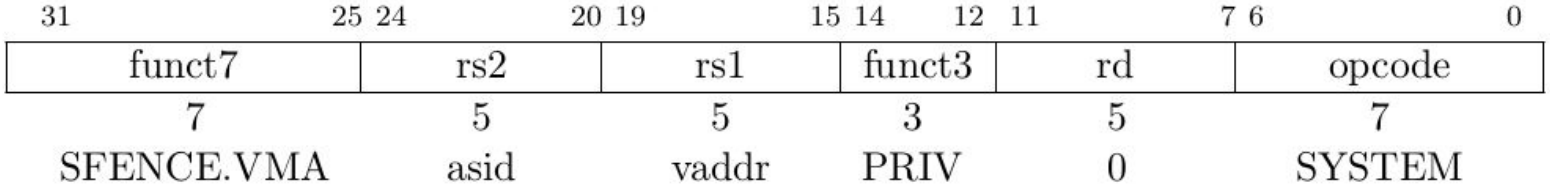
HW Remote TLB Invalidation

- Current linux-riscv use software interrupt to flush remote harts' TLB with the local flush instruction
- Hardware remote TLB invalidation broadcast (supported by 860, 910)
- Parameters:
  - VA
  - ASID
  - VMID
  - ***Enough ?***
  - ***ASID's S1.PGD.PPN (stage1/level1 translation, mm->pgd)***
  - ***VMID's S2.PGD.PPN (stage2/level2 translation)***
- *First level Translation* abbrev *to S1*
- *Second level Translation* abbrev *to S2*
- *Physical Page Number abbrev to PPN*

Current sfence.vma definition

- sfence.vma rs1 = vaddr, **rs2 = asid**
- hfence.vvma rs1 = vaddr, **rs2 = asid**
- hfence.gvma rs1 = gaddr, **rs2 = vmid**

4.2.1 Supervisor Memory-Management Fence Instruction



Proposed sfence.vma

- sfence.vma rs1 = vaddr, **rs2 = mode:ppn:asid**
- hfence.vvma rs1 = vaddr, **rs2 = mode:ppn:asid**
- hfence.gvma rs1 = gaddr, **rs2 = mode:ppn:vmid**

mode:1bit	local	broadcast
id	asid/vmid	asid/vmid
ppn	0 (ignored)	S1/S2.PGD.PPN (relate to asid/vmid)

Why we need S1/S2.PGD.PPN ?

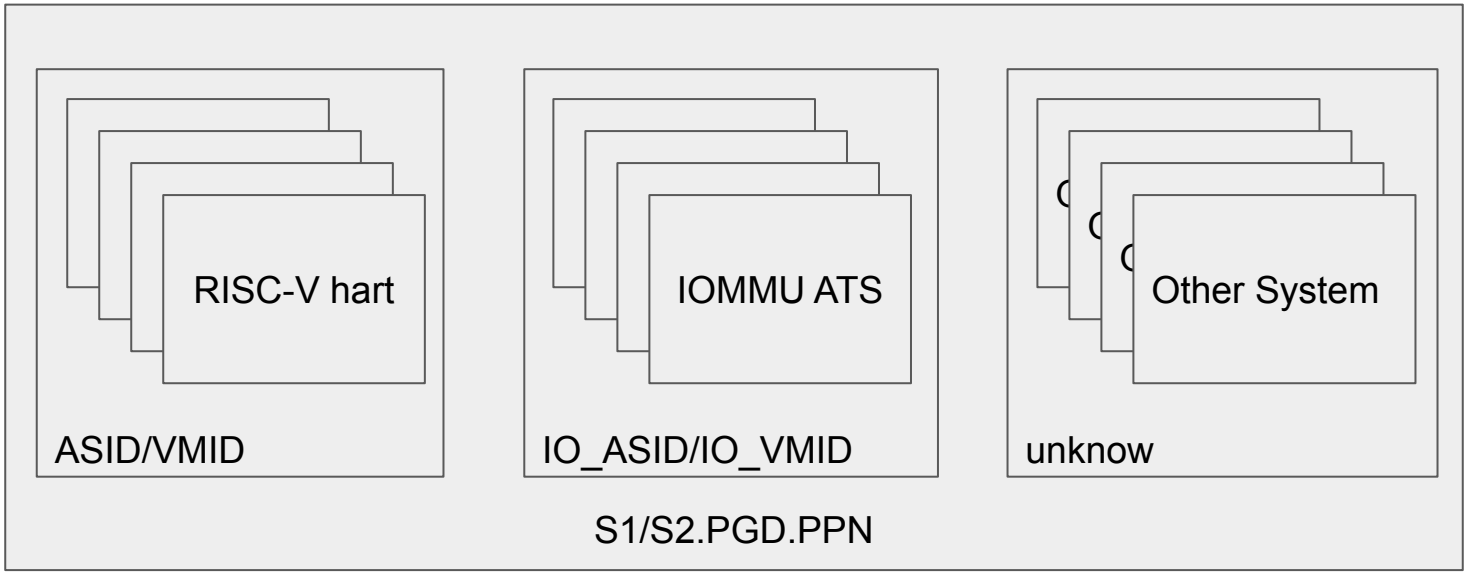
ASID/VMID is a window technology to identify different address spaces in TLB and it’s hard to maintain CPU & IOMMU with the same ID pool.

- ASID/VMID of CPU SMP system are dynamically changed during process/VMs’ context switch, it’s a window of the address spaces.
- ASID/VMID of IOMMUs are binded at devices’ slots and they needn’t be changed as CPU SMP system.

- **S1/S2.PGD.PPN** is unique identifiers for address spaces and their life cycle is the same as the address space.

Window ID Jurisdiction

ASID/VMID window technology is only suitable for local symmetric system scenarios. But for cross-system communication, we need a unique identifier instead of window/temporary identifier.



S1/S2.PGD.PPN for sfence.vma broadcast mode

- Use broadcast invalidate protocol (eg: DVM) to transfer “vaddr, asid, vmid, **S1.PGD.PPN, S2.PGD.PPN**” on interconnect
  - DVM is designed to operation read only cache, eg: TLB, Icache, BR-predict-buf
  - Normally DVM only use “vaddr, asid, vmid”
  - **S1/S2.PGD.PPNs will cause more transfers for one DVM operation transaction**
- Difference between Hypervisor on/off
  - When hypervisor off, only transfer VA, ASID, PGD.PPN on interconnect
  - When hypervisor on:
    - Guest OS:
      - GVA, ASID, S1.PGD.PPN (by software)
      - current VMID & S2.PGD.PPN (by hardware automatically)
    - Host OS:
      - Maintain host page table: VA, ASID, PGD.PPN
      - Maintain S2 page table: Use Mode:1 with GPA, VMID, S2.PGD.PPN

Fake Code:

- Loop check
  - Intel:
    https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/iommu/intel-svm.c?h=v5.3
    .invalidate_range = intel_invalidate_range
    -> intel_flush_svm_range
    -> intel_flush_svm_range_dev
    -> qi_submit_sync
    -> while(== QI_DONE) cpu_relax()
  - AMD:
    https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/iommu/amd_iommu_v2.c?h=v5.3
    .invalidate_range = mn_invalidate_range,
    -> amd_iommu_flush_page
    -> wait_on_sem
    -> while(*sem == 0) udelay(1);
- wait irq signal, io_schedule
  https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/gpu/drm/radeon/radeon_mn.c?h=v5.3

```
flush_tlb_page(vma, addr) {
    asid = cpu_asid(vma->vm_mm);
    ppn = PFN_DOWN(vma->vm_mm->pgd);

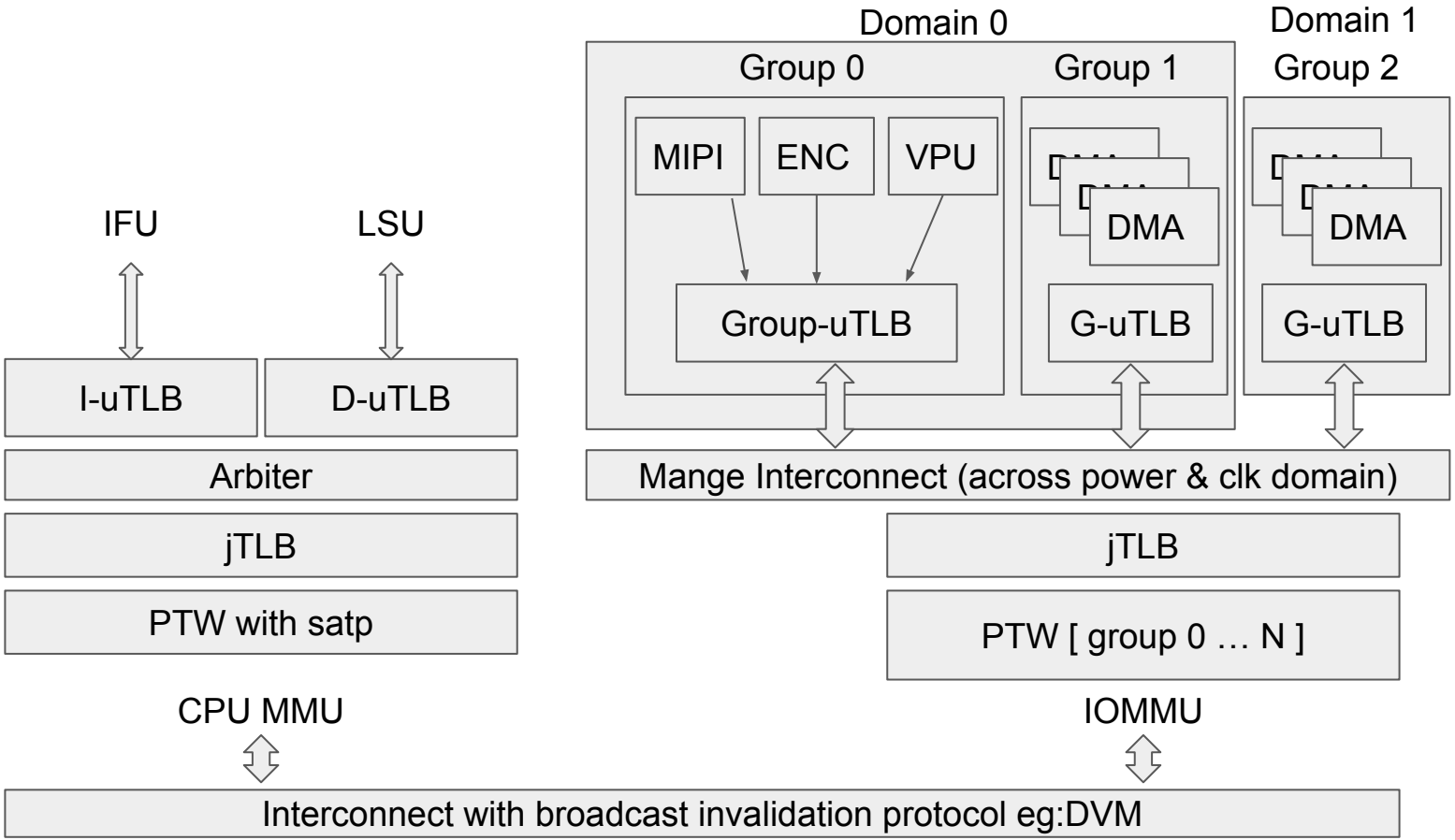
    sfence.vma (addr, 1|PPN_OFFSET(ppn)|asid); // 1. start

    while(sstatus:TLBI) if (time_out() > 2ms) break; // 2. loop check

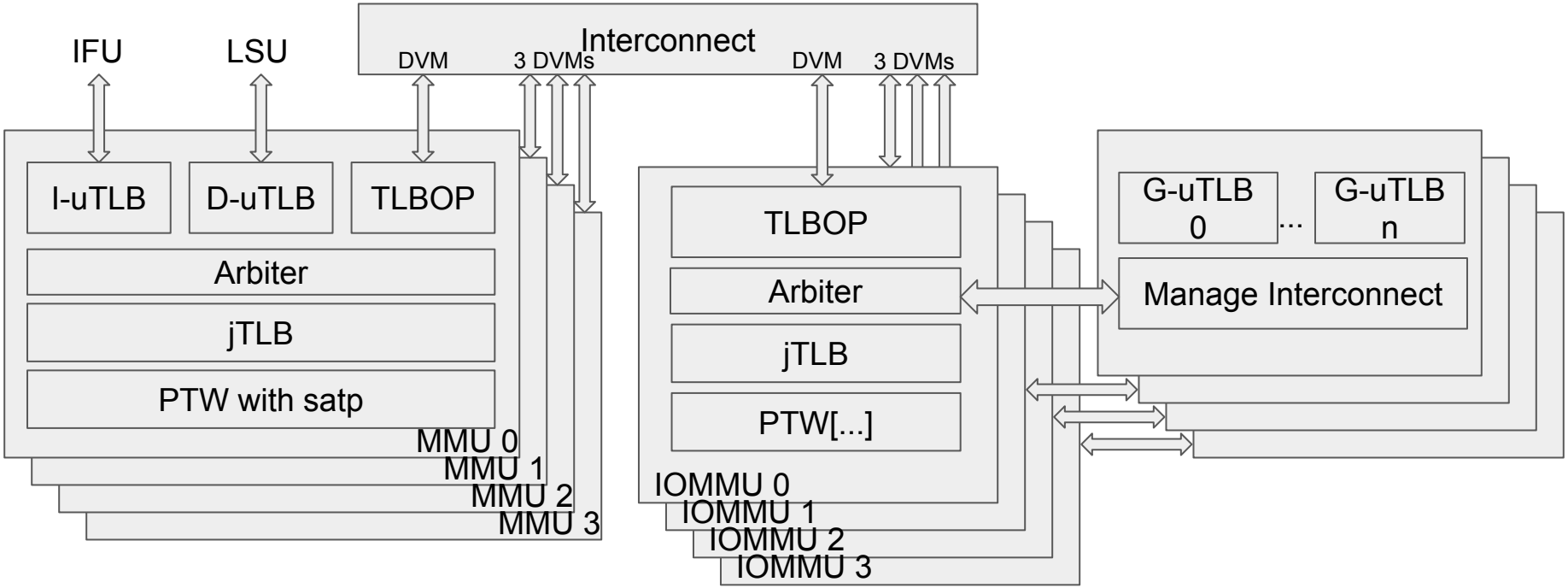
    while (sstatus:TLBI) {
        …
        set sstatus:TLBI;
        wait_TLBI_finish_interrupt(); // 3. wait irq signal, io_schedule
    }
}
```

Introduce a light implementation of IOMMU

From MMU to IOMMU



Expand to SMP CPUs & IOMMUs



Concepts of GID, DID, PGD.PPN

- Group-uTLB is the same with linux iommu group concept and it’s the minimum granularity for namespace isolation
- Each G-uTLB has its own PGD.PPN register
- Each G-uTLB has its own interrupt for page fault request
- jTLB duty to PTW and page fault interrupt control and response DVM request
- jTLB use DID (Domain ID) to identify different entries’ address space

GID (index)	DID	PGD.PPN
0	0	0x4000
1	0	0x4000
2	2	0x8000
3	1	0x18000
4	3	0x28000
5	3	0x28000
...	...	

Group Mapping regs[...]

TLB invalidate mechanism

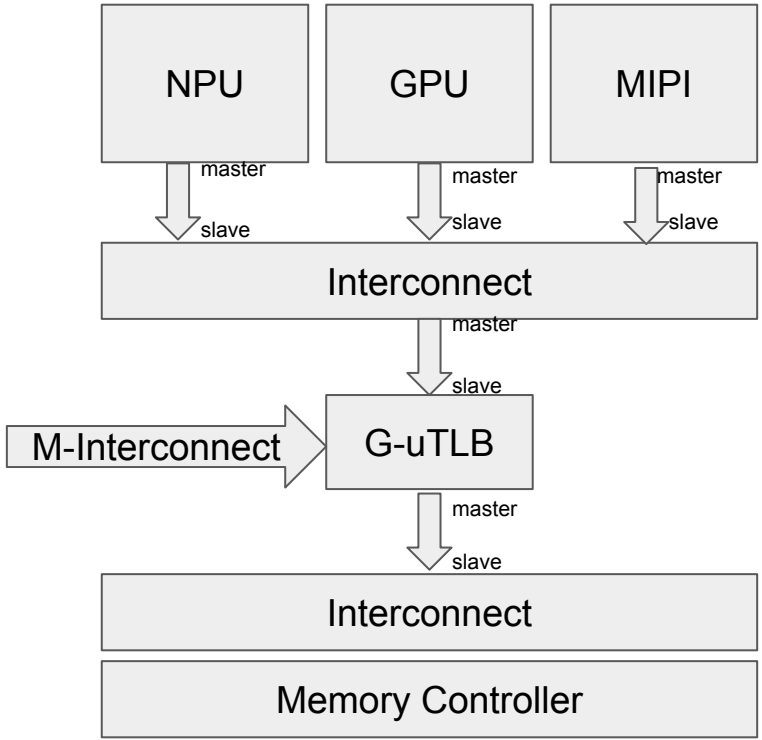
Use sfence.vma.b to invalidate the virtual address' TLB entry of system, eg: sfence.vma.b rs1=VA rs2=ASID + PGD.PPN

- ASID is used by RISC-V hart
- PGD.PPN is used by IOMMU
- IOMMU directly use PGD.PPN to detect Domain ID in Group Mapping Regs[...], so the number of groups is limited (max:64) to reduce hardware resource.
- Use DID + VA to invalidate TLB entry:

VA.tag	DID	PPN	Attributes

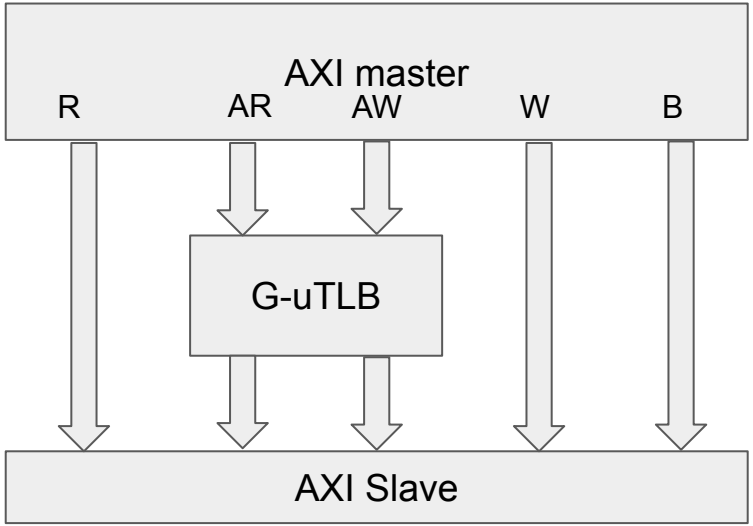
G-uTLB Detail:

- G-uTLB don’t care name space, we don’t provide any sideband signals. (ref: ARM SMMU)
- Only support first translation, no virtualization
- M-Interconnect help to across clk & power domains



Translation Principle eg: AXI 5 channels

Read Address Channel	Write Address Channel
ARADDR	**AWADDR**
ARVALID	**AWVALID**
ARREADY	**AWREADY**
...	...
ARID	AWID
ARLEN (bypass)	AWLEN (bypass)
ARSIZE (bypass)	AWSIZE (bypass)
ARBURST (bypass)	AWBURST (bypass)
...	...



R: Read Data Channel
AR: Read Address Channel
AW: Write Address Channel
W: Write Data Channel
B: Write Response Channel

Generic Page Fault Issues

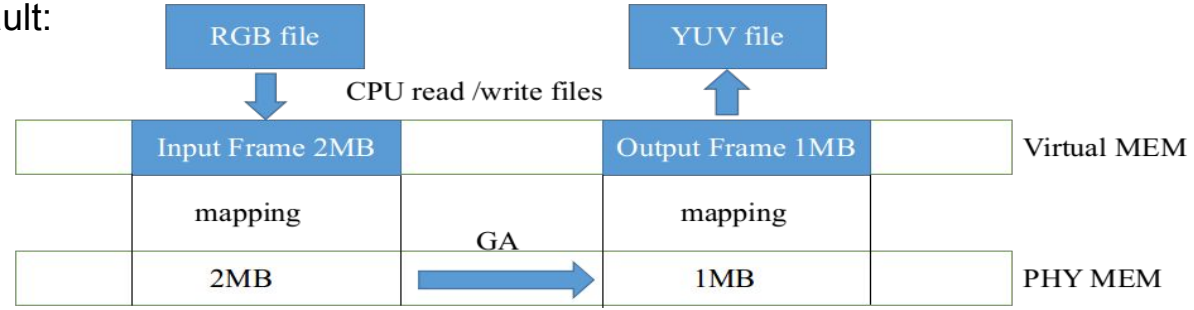
```
➜  linux-next git:(linux-next-for-v5.4) ✗ grep handle_mm_fault drivers/iommu -r
drivers/iommu/intel-svm.c:              ret = handle_mm_fault(vma, address,
drivers/iommu/amd_iommu_v2.c: ret = handle_mm_fault(vma, address, flags);
```

- Determine mm with Interrupt context
- Which task belongs to ?
  (In general arch do_page_fault flow, some issues depends on current task, eg: set_current_state, perf_sw_event, signal)
- When do_page_fault error, we need to signal the task (Just like common processes in linux)

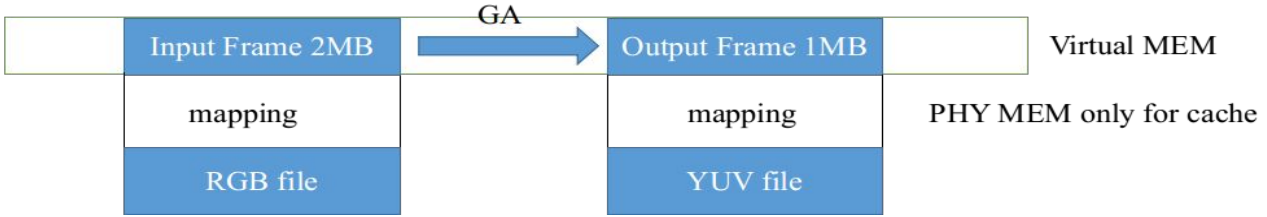
Reference: amd_iommu change get_user_pages to handle_mm_fault
https://lore.kernel.org/lkml/1415830228-7844-2-git-send-email-jbarnes@virtuousgeek.org/

convert a RGB file to YUV file

No handle_mm_fault:



handle_mm_fault:



- Memory Saving - 3MB v.s. ??KB
- Performance Improving - working in parallel with page granularity

Customize TLB Prefetch model with Perf Tracer

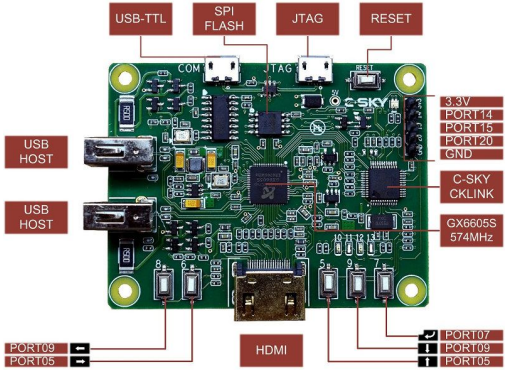
- Unlike CPU local random access, DMA access generally lacks locality, but the order of access is regular.
- So TLB prefetch is a very important feature to prevent translation performance bottleneck
- We provide a variety of alternative prefetch models for each G-uTLB to meet address access characteristics of different groups, then we could minimize the hardware resources of G-uTLB
  - **PMU counter could determine G-uTLB miss/hit rate**
  - **PMU tracer record all translations of G-uTLB**

Usage scenarios:

GX6605S SIP 64MB DDR

DVB-S2+H.264 1080P HD

https://c-sky.github.io/docs/gx6605s.html



- Reserve 32MB for Frame Buffer
- Honestly, CMA wasn't much help
- ***Hope our IOMMU solution could help this kind of product***



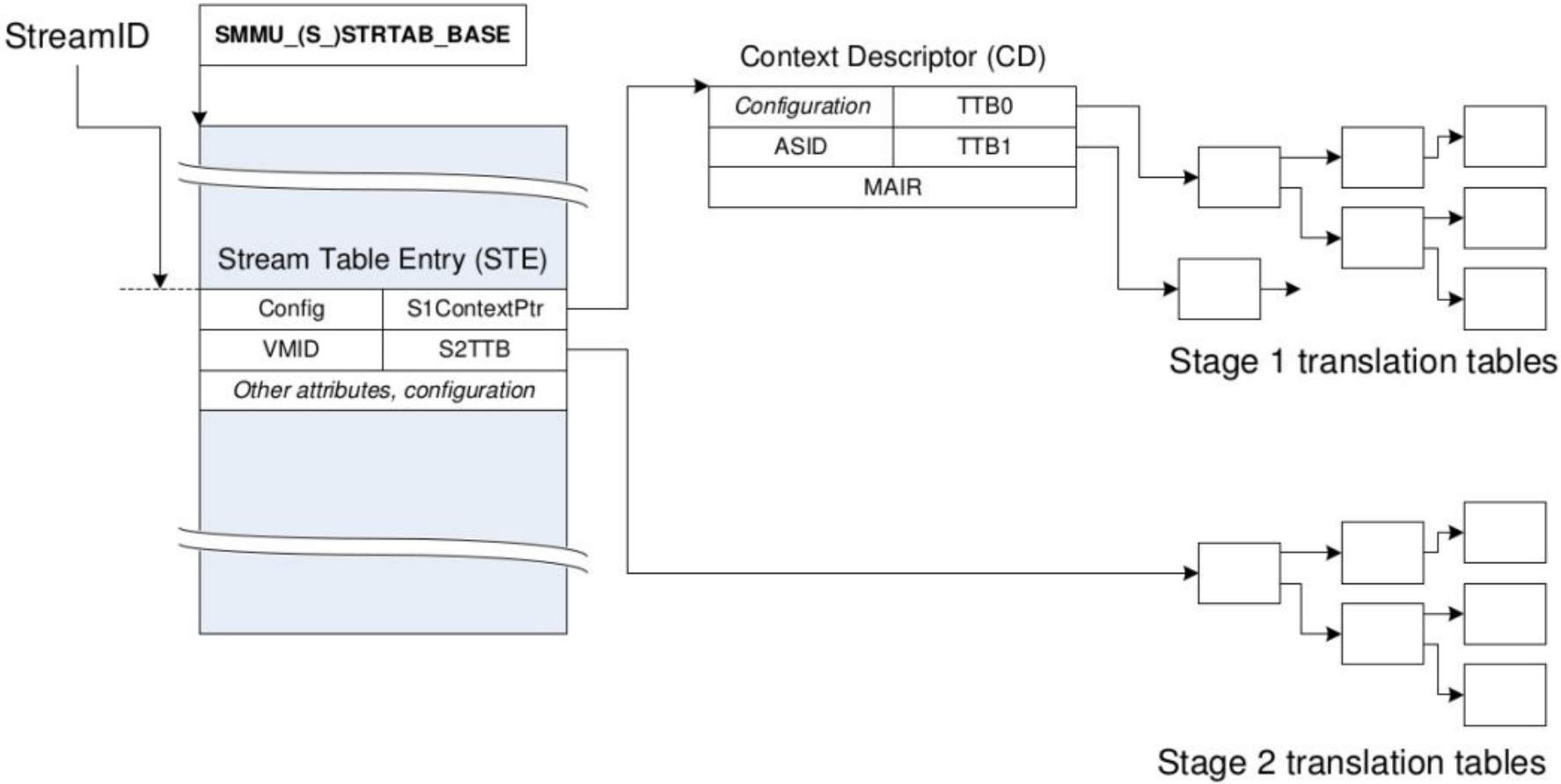
Is **S1/S2.PGD.PPN** suitable for millions of devices (PCI ATS & Virtualization)

?

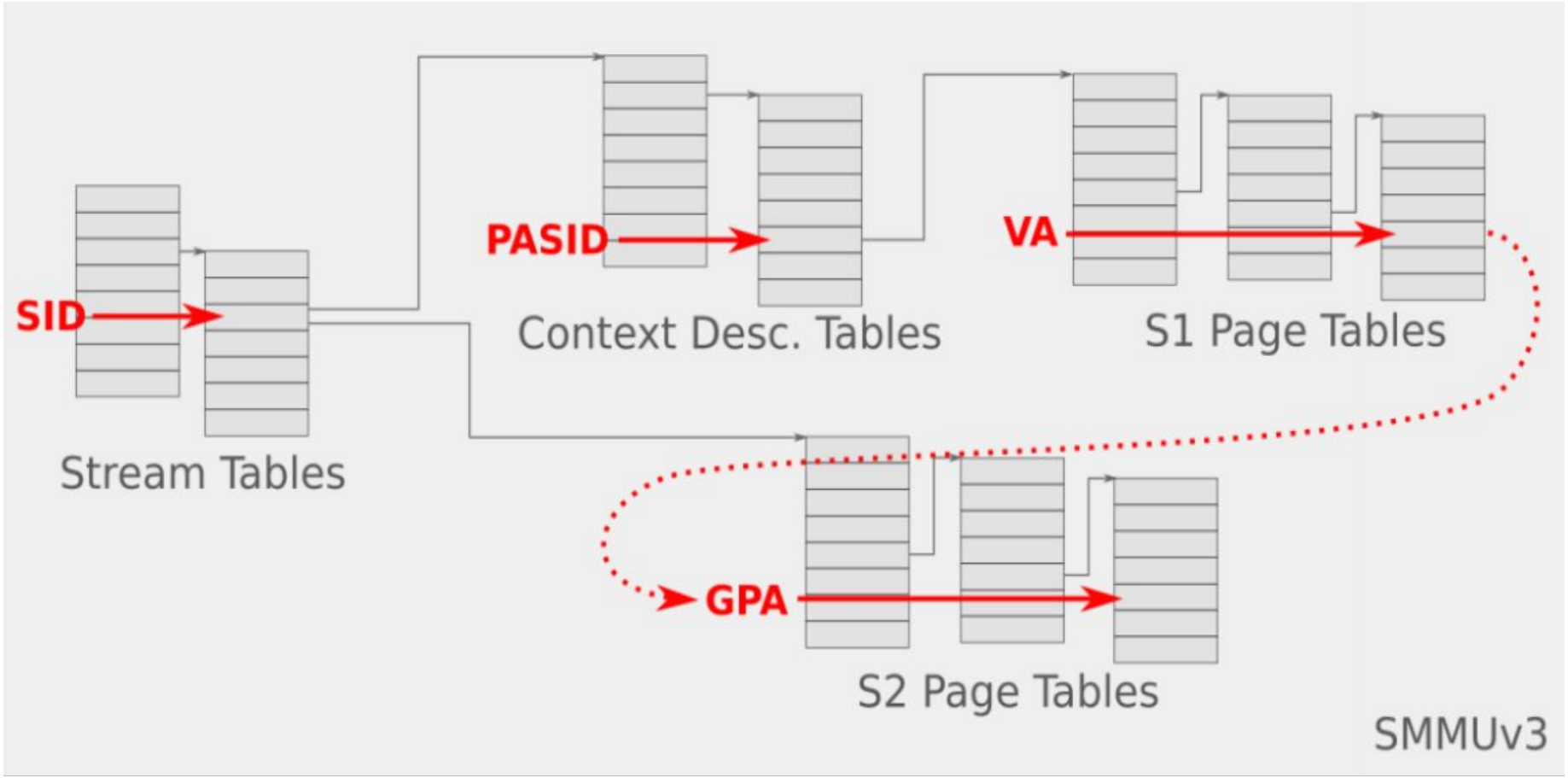
StreamID/RID/SubstreamID/PASID Concept

<table>
<tr><td rowspan="2">StreamID:16-31(max)</td><td colspan="3">PCI RID (StreamID: 0-15)</td></tr>
<tr><td>Bus:8-15</td><td>device:3-7</td><td>Func:0-2</td></tr>
<tr><td></td><td colspan="3">Intel use RID to setup root_context Table</td></tr>
<tr><td colspan="4">Arm use streamID to setup Sream Table</td></tr>
</table>

<table>
<tr><td>PASID/SubstreamID:0-19</td></tr>
<tr><td>Intel use PASID to index PASID Directory/Table</td></tr>
<tr><td>Arm use substreamID to index Context Descript Table</td></tr>
<tr><td>RISC-V use xxxID to index XXX Table</td></tr>
</table>



Reference: Arm System Memory Management Unit Architecture Specification 3.2



Reference: Arm System Memory Management Unit Architecture Specification 3.2

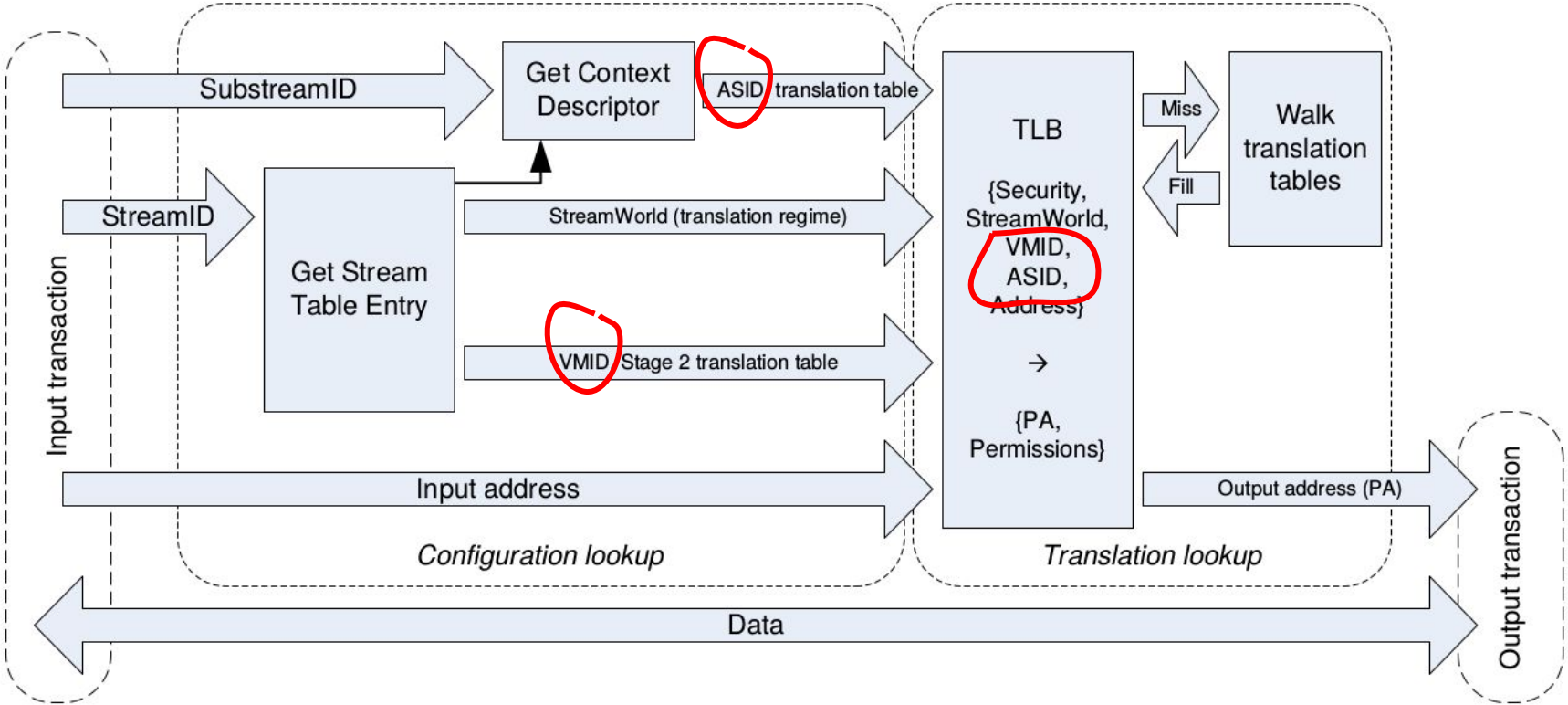


Figure 10: Configuration and translation lookup sequence

Reference: Arm System Memory Management Unit Architecture Specification 3.2

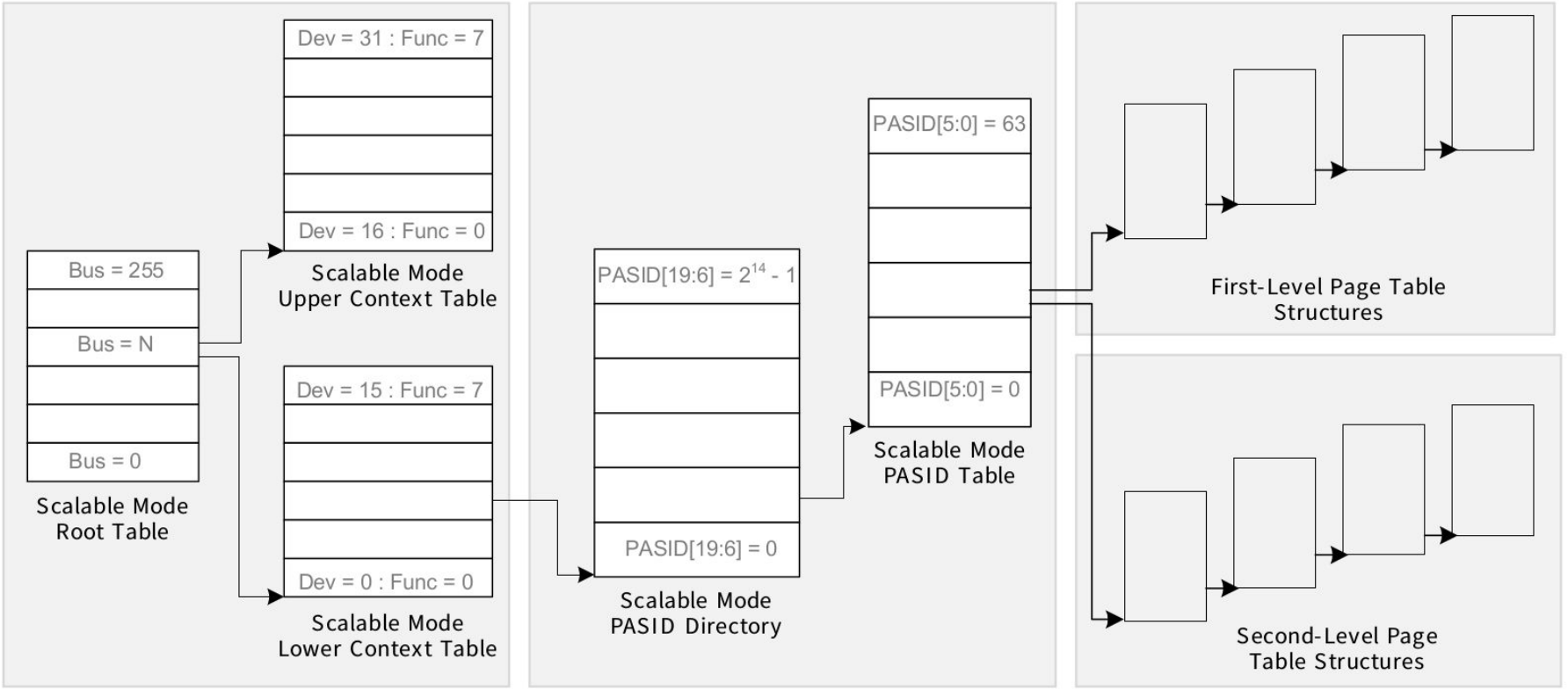


Figure 3-8. Device to Domain Mapping Structures in Scalable Mode

Reference: Intel Virtualization Technology for Directed I/O Architecture Specification

Guess their IOTLB entry ?

Guest VA.tag	Host PFN	DID/VMID	PASID/ASID	Attributes
Host VA.tag				
Guest PA.tag				

TLB broadcast invalidation

Actually ASID/VMID & IO_ASID/IO_VMID is hard to maintain in a unified way.
(Intel: DID=IO_VMID PASID=IO_ASID)

- ASID/VMID of CPU SMP system are dynamically changed during process/VMs' context switch, it's a window of the whole address spaces.
- ASID/VMID of IOMMU system are binded at devices' slots and they needn't be changed as CPU SMP system
- Use sfence.vma.b to broadcast **S1/S2.PGD.PPN** on bus for these different systems

Solution 1:

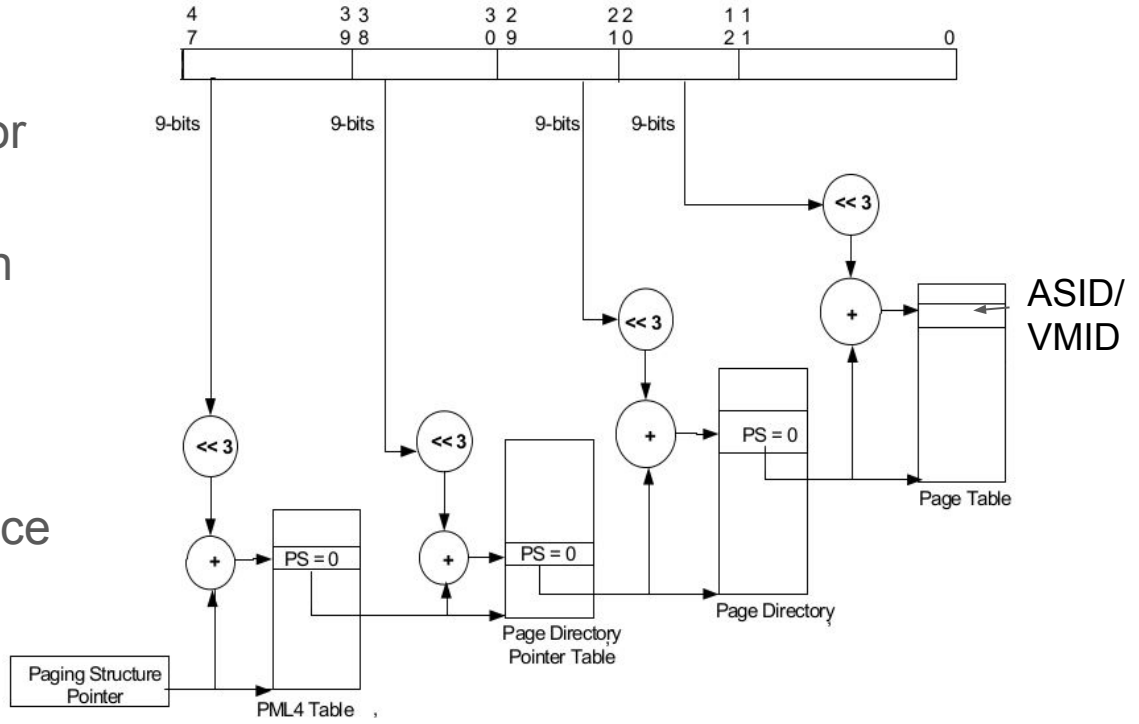
Use S1/S2.PGD.PPN instead of ASID/VMID or PASID/VMID

- RISCV64 ASID: 16-bits -> 34 bit (48 bit Physical Address Width - 256TB)
- VMID is similar

  The disadvantage is obvious that increasing the ASID tag width of each TLB entry are very expensive for chip.

Solution 2:

- Whether S1.PGD.PPN->ASID/PASID or S2.PGD.PPN->VMID/DID is one-to-one correspondence in TLB
- Create translation tables for PPN -> ID
- Add TLB Buffer for performance



Conclusion

- Suggest sfence.vma broadcast mode to RISC-V
  - Add S1/S2.PGD.PPN in addition to ASID & VMID
  - Add async mechanism (optional)
- It’s suitable for light IOMMU implementation
  - Perf Tracer is very important feature for IOMMU
- It should be suitable for millions of devices (PCI ATS & VM)



T-HEAD

Thank you

Any Questions ?